

Fiori Elements 開發課程 14 (延伸篇) : 訊息模型——RAP 錯誤訊息怎麼呈現在畫面上

環境：BTP ABAP Environment Trial (沿用 fe01 的 `fe01_connection_test` 專案；`ZI_RC05_NOTE` 新增一條 Validation，套件 `ZRAPCLOUD`，需要傳輸請求)

Lecture

這一課要補的洞

課程總覽時 grep 了 fe01 ~ fe12 全部講義，`MessageToast` 只出現在 fe05 / fe06 的自訂 **Controller Extension** 示範文字 (例如「已掛上標準 Demo Travel App」這種教學提示)，整門課零處觸及「**RAP Validation** 失敗，畫面上到底長什麼樣」。這是一個很不該漏掉的主題——ABAPER 寫 RAP 後端邏輯，`reported` / `failed` 這兩個回應參數存在的唯一理由，就是要讓終端使用者在畫面上看到「為什麼存不進去」；如果從沒看過真實呈現效果，很難判斷自己寫的訊息文字、標記的欄位對不對。

這一課分兩段：**Part A** 完全不用改 ABAP，示範框架內建的必填欄位錯誤；**Part B** 新增一條自訂 Validation，示範「你自己寫的商業規則訊息」怎麼跑出來——這才是 RAP `reported` 機制真正的價值所在。

⚠️ ⚠️ 動手前必看：fe04 已經把 Object Page 換成唯讀的 Custom Page，這兩堂課會打架

這是實測踩到的坑，寫進來提醒後面的人：fe04 那一課把 `NoteObjectPage` 從標準範本換成一個唯讀的 Custom Page (`NoteDetail.view.xml`，欄位全部是 `ObjectAttribute` / `FormattedText`，沒有任何一個是可編輯的 `Input`)，而且 fe04 自己的講義就寫明了這個取舍：「換成 Custom Page 之後，Create / Edit / Delete 這一整套 UI 沒有了」。

`fe01_connection_test` 是同一個專案，fe04 的改動會一路留到現在——這代表 **Part A** / **Part B** 底下「點 **Create**」之後看到的畫面，預設會是 fe04 那個唯讀畫面，完全沒有欄位可以輸入，不是這一課新增的 Validation 出了問題。

動手操作前，先把 `webapp/manifest.json` 的 `routing.targets.NoteObjectPage` 切回標準範本 (這正是 fe04 講義裡「改之前」的版本)：

```
"NoteObjectPage": {
  "type": "Component", "id": "NoteObjectPage",
  "name": "sap.fe.templates.ObjectPage",
  "options": { "settings": { "editableHeaderContent": false, "contextPath": "/Note" } }
}
```

存檔，livereload 會自動反映，Object Page 就會變回有真正 `Input` 欄位、有 Save 按鈕的標準表單。這一課測完之後，如果想恢復 fe04 的 **Custom Page** 效果，把這段改回 fe04 講義「改之後」的版本 (`sap.fe.core.fpm + viewName`) 即可——兩邊的檔案都還在，只是 `manifest.json` 指向哪一個。

這個插曲本身也是一個值得記住的教訓：同一個前端專案被多堂課輪流疊加修改時，後面的課程不能假設「某個畫面還是最初範本產生的樣子」——寫新講義前，最好先確認一下專案當下的真實狀態，不能只憑之前哪一

課教過什麼就推論現在長什麼樣。

Part A (零後端改動) : ⚠ 實測推翻原本預期——`field(mandatory)` 其實不會擋存檔

Eclipse 裡的 `ZI_RC05_NOTE` (Behavior Definition , 不是本機檔案 , 是 **Eclipse ADT** 連線到 **SAP** 系統裡的真正物件——`zi_rc05_note.bdef.abap` 只是這個 git repo 存放快照用的檔名慣例 , 不是你要在 Eclipse 打開的東西 ; 要在 Eclipse 找到它 , 用 `Ctrl+Shift+A` 搜尋 `ZI_RC05_NOTE` 最快) 早就有這一行 :

```
field ( mandatory ) title;
```

這一段最早的草稿寫著「這個標記會讓 **Fiori Elements** 前端擋下空白 **Title**」——實測結果直接推翻了這個預期 : Title 留空按 Create , 畫面上完全沒有紅框、沒有星號、沒有任何必填提示 , 甚至存檔動作真的送出去 , **Title** 是空的那筆資料照樣存進資料庫 (實測截圖 : `note_id = 'RC05TEST03'` , Title / Content 兩欄都顯示 - , 畫面跳出 `Object saved`) 。連正在輸入的草稿畫面 (`IsActiveEntity=false`) Title 欄位也只是普通的藍框 (游標焦點) , 沒有任何必填樣式。

查證官方文件 `ABENBDL_FIELD_CHAR` 才找到真正原因 , 而且文件講得非常明確 :

`field(mandatory)`——"Defines that it is mandatory to enter values into the specified fields before persisting them on the database. These fields are marked as mandatory on the user interface in an OData scenario. **However, there is no runtime check for mandatory fields and no runtime error occurs if a mandatory field is not filled. If a runtime check is required, the application developer should implement it using a validation on save.**"

關鍵在這裡 : `mandatory` 這個關鍵字有兩種寫法 , 行為完全不同 :

寫法	官方文件怎麼說	實際效果
<code>field (mandatory)</code>	「標記為 UI 提示用 , 沒有執行期檢查 , 欄位空白也不會報錯」	純粹是給消費端 (理論上) 的一個提示旗標 , 這個 Fiori Elements 版本連提示樣式都沒有明顯呈現 , 實測完全不擋存檔
<code>field (mandatory:create)</code>	「Static field attribute.....如果 RAP BO 消費端沒填這個欄位就嘗試建立實例 , 會發生執行期錯誤」	這才是真正會擋下存檔的寫法

`ZI_RC05_NOTE` 目前用的是前者 (`mandatory` , 沒有 `:create`) , 這正是官方文件白紙黑字說明「不會有執行期檢查」的那一種——你這次實測到的 , 不是這個環境的 **bug** , 是官方文件本來就講明的預期行為 , 只是原始草稿誤把兩種寫法當成同一件事。

想看到真正被擋下的效果 , 要改成 `mandatory:create`

回 Eclipse (`Ctrl+Shift+A` 搜尋 `ZI_RC05_NOTE` 打開這個 Behavior Definition) , 把這一行 :

```
field ( mandatory ) title;
```

改成 :

```
field ( mandatory:create ) title;
```

存檔、啟用，回到 VS Code 重新測一次 (Title 留空按 Create)。

✅ 實測確認：**mandatory:create** 會讓 Fiori Elements 改跳出 **Create Dialog** (快速建立對話框)，不是直接進 **Object Page**

改成 **mandatory:create** 後，List Report 點 **Create**，畫面不會像之前那樣直接進到完整的 Object Page，而是先跳出一個小對話框，只問 **note_id** (**readonly:update**，建立當下必填) 跟 **title** (現在是 **mandatory:create**) 這兩個欄位——**content** 不是 mandatory，不會出現在這裡；**title** 欄位帶著紅色星號 *，是真正的必填視覺提示 (跟 Part A 前半 **mandatory** 完全沒有任何提示的畫面形成鮮明對比)。

原因：**note_id** 是使用者自己輸入的 Key (不是框架自動編號)，**title** 現在也是「建立當下就一定要有值」——Draft 連一筆空白草稿都沒辦法在後端先建出來 (會撞到官方文件講的「a runtime error occurs」)，所以框架乾脆先跳對話框把這兩個非有不可的欄位問清楚，確定有值了才真的送出 Draft 建立請求。這代表 **mandatory** 跟 **mandatory:create** 不只是「擋不擋存檔」的差異，連 UI 呈現方式都完全不同 (一個直接進全表單 Object Page、一個先跳小對話框)——這是動手做才會發現的細節，光看語法文件猜不到。

填完 **note_id** / **title**，按 **Continue**，會建立草稿並正常導到完整 Object Page，**content** 欄位在那裡才看得得到、才能繼續測 Part B。

這一步的教學重點

- **field(mandatory)** (不帶 **:create**) 只是一個消費端的提示旗標，這個環境沒有把它落實成畫面上的必填樣式，也完全沒有執行期擋控——寫 RAP BDEF 時很容易望文生義，以為「mandatory」四個字母出現就代表一定會被擋下，這是本課實測踩到、也親手推翻的一個具體案例
- 真正要「值一定要填、不填就報錯」，要嘛用 **mandatory:create** (單欄位、沒有跨欄位邏輯的簡單情境，訊息文字是系統制式文字、不能自訂)，要嘛自己寫 Validation (on save 檢查 **IS INITIAL**，可以有更複雜的條件、也能自訂訊息文字)——這正好銜接到下面 Part B 要教的機制

Part B (新增一條 Validation)：自訂商業規則訊息怎麼寫、怎麼呈現

title / **content** 目前沒有任何跨欄位的邏輯關聯。這一課新增一條示範用的商業規則：「**Title** 不可以跟 **Content** 完全相同」(模擬真實場景常見的「使用者複製貼上打錯」防呆)——這種跨欄位比對，**field(mandatory)** 或資料庫層的長度限制都做不到，一定要靠 RAP Validation。

官方語法查證：**%msg** 怎麼填

查證官方文件 **ABAPDERIVED_TYPES_MSG** (ABAP Cloud) 確認：**%msg** 是 **REPORTED** 衍生型別的一個欄位，型別是 **IF_ABAP_BEHV_MESSAGE**；不需要自己寫 Message Class，直接呼叫 Handler 類別繼承來的 **new_message_with_text()** 方法就能組出一個臨時訊息：

```
%msg = new_message_with_text(  
    severity = if_abap_behv_message=>severity-error  
    text      = 'Validation failed' )
```

官方範例同時示範用 `%element-<欄位名> = if_abap_behv=>mk-on` 標記「這個錯誤該貼在哪個欄位上」——這會讓 Fiori Elements 在 Object Page 對應欄位旁邊也顯示紅色錯誤標記，不是只有一個籠統的訊息彈窗。

這一課選 `new_message_with_text`、不用 `Message Class` 的原因

RAP 訊息還有另一條路：用 **T100 Message Class** ([SE91](#) / ADT 建立，訊息文字支援多語系、可重複使用) 搭配 `NEW zcx_...` 拋出。這是比較「正式」的做法，但要多建一個 `Message Class` 物件。這一課為了讓新增的後端改動維持最小，選擇 `new_message_with_text`——文字直接寫死在 ABAP 程式碼裡，**沒有多語系翻譯的好處，也沒辦法被別的程式重複引用同一則訊息**，正式專案如果同一則訊息會被多處引用、或需要支援多國語言，建議改用 `Message Class`；這裡純粹是為了教學 / 示範選擇成本最低的做法。

BDEF 修改：加一行 `validation`

回頭在 Eclipse 打開 `ZI_RC05_NOTE` (Behavior Definition，`Ctrl+Shift+A` 搜尋最快，**不是**去找一個叫 `zi_rc05_note.bdef.abap` 的檔案——這個檔名只在這個 `git repo` 的本機快照裡存在，Eclipse ADT 認的是物件名稱 `ZI_RC05_NOTE`)，在既有 `field(...)` 那幾行後面加一行：

```
managed implementation in class zbp_i_rc05_note unique;
strict ( 2 );
with draft;

define behavior for ZI_RC05_NOTE alias Note
persistent table zrc05_note
draft table zrc05_note_d
lock master
total etag changed_at
etag master local_changed_at
authorization master ( none )
{
    create;
    update;
    delete;

    field ( readonly : update ) note_id;
    field ( readonly )          changed_at, local_changed_at;
    field ( mandatory )         title;

    validation validateTitleContent on save { field title, content; }

    draft action Activate optimized;
    draft action Discard;
    draft action Edit;
    draft action Resume;
    draft determine action Prepare;
}
```

`on save { field title, content; }`——延續 rc03 學過的觸發時機規則：只要 `title` 或 `content` 任一欄位在這次交易裡被改過，Save (Draft 的 `Activate`) 時就會觸發這條 Validation。

Local Types : Handler Method 完整內容

`ZBP_I_RC05_NOTE` 這個 Behavior Pool 類別的全域殼目前完全是空的 (`zbp_i_rc05_note.clas.abap` 只有 `CLASS ... ENDCLASS.` 兩行) ——這是 rc05 建立至今第一次要幫它補上 Local Types 內容。在 Eclipse 開啟 `ZBP_I_RC05_NOTE`，切到 **Local Types** 分頁 (跟 Global 定義 / 實作是分開的區塊)，貼入：

```
CLASS lhc_note DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS validateTitleContent FOR VALIDATE ON SAVE
      IMPORTING keys FOR Note~validateTitleContent.
ENDCLASS.

CLASS lhc_note IMPLEMENTATION.

  METHOD validateTitleContent.
    READ ENTITIES OF zi_rc05_note IN LOCAL MODE
      ENTITY Note
        FIELDS ( note_id title content )
        WITH CORRESPONDING #( keys )
        RESULT DATA(lt_note).

    LOOP AT lt_note INTO DATA(ls_note).
      IF ls_note-title = ls_note-content AND ls_note-title IS NOT INITIAL.

        APPEND VALUE #( %key = ls_note-%key ) TO failed-note.

        APPEND VALUE #( %key = ls_note-%key
          %msg = new_message_with_text(
            severity = if_abap_behv_message=>severity-error
            text      = |Title and Content must not be identical
              (Note { ls_note-note_id })| )
          %element-title = if_abap_behv=>mk-on
          %element-content = if_abap_behv=>mk-on )
          TO reported-note.
      ENDIF.
    ENDLOOP.
  ENDMETHOD.

ENDCLASS.
```

- **FOR VALIDATE ON SAVE**——官方現行語法 (這個 Cloud 環境沒有 rap05 / rap06 那種 On-Premise 舊系統被迫用 obsolete `FOR VALIDATION` 的問題，rc03 已經驗證過)，跟 rc03 的 `validateStatus` 寫法完

全同一套模式，只是這次是單一實體（沒有 Composition 子實體），`%key / failed-note / reported-note` 直接用，不用 `%tky / %own`

- **READ ENTITIES ... IN LOCAL MODE**——rc03 已經教過的固定寫法，Validation Handler 裡讀取當下要驗證的實例資料
- **%element-title / %element-content** 都標記 **mk-on**——因為這是一個橫跨兩個欄位的規則，兩個欄位都該顯示錯誤標記，不是只標一個
- 字串模板 **|Title and Content must not be identical (Note { ls_note-note_id })|**——把觸發規則的 `note_id` 直接嵌進訊息文字，讓使用者存好幾筆同時失敗時能分清楚是哪一筆

⚠️ ⚠️ 實測踩坑：**LOOP AT ... WHERE title = content** 編譯失敗——**Field "CONTENT" is unknown.**

這是原始草稿的語法錯誤，Eclipse 語法檢查直接擋下來（`ZBP_I_RC05_NOTE` 的 `LHC_NOTE` 類別，`validateTitleContent` 方法第 17 行）：原本寫的是

```
LOOP AT lt_note INTO DATA(ls_note)
  WHERE title = content AND title IS NOT INITIAL.
```

錯誤指在 `content` 這個字上，訊息是 **Field "CONTENT" is unknown.**——但 `title` 完全沒有被標記錯誤，這個不對稱的現象是關鍵線索。

原因：ABAP 的 **LOOP AT itab WHERE comp = dobj** 這個古典（非括號式）語法，**WHERE** 子句左邊的 `comp` 會被解析成 `itab` 這張表本身的欄位名稱，但右邊的 `dobj` 預期是一個表外部的資料物件（變數 / 字面值），不會被當成同一張表的另一個欄位去解析。這一課的寫法 `title = content` 剛好就是「拿同一張內部表的兩個欄位互相比較」，左邊 `title` 順利解析成欄位，右邊 `content` 因為語法規則不會被當作欄位查找，系統作用域裡也沒有一個叫 `content` 的獨立變數，才報「Field CONTENT is unknown」——這個錯誤訊息本身有點誤導，讓人以為是打錯欄位名，實際上是語法結構本身不支援這種寫法。

修法：拿掉 **LOOP ... WHERE**，改成先無條件 **LOOP**，迴圈內用 **IF** 明確透過工作區（`ls_note-title / ls_note-content`）比較兩個欄位——這是上面已經更新過的正確版本，**WHERE** 子句完全拿掉，判斷邏輯搬進 **IF** 區塊，其餘的 **APPEND / %msg** 內容不變。這個限制不是這個環境特有的 **bug**，是 **ABAP** 語言本身「**LOOP ... WHERE** 只能拿欄位跟外部變數比較，不能拿同一張表的兩個欄位互相比較」的通用規則——這一課因為剛好寫了一個「比較同一筆資料的兩個欄位」的商業規則，正好踩到這個平常不太會遇到的語法邊界。

存檔、啟用（先啟用 **BDEF**，再啟用 **Behavior Pool** 類別）。

動手操作：在真正的 VS Code App 裡觸發這則訊息

1. 沿用 `fe01_connection_test`，**npm start**（如果已經在跑就直接切過去）
2. List Report 點 **Create**
3. **Title** 跟 **Content** 都填一模一樣的文字（例如都打 `test`）
4. 按 **Create**（儲存 / Activate Draft）
5. 預期畫面：儲存被擋下，畫面上出現訊息（Fiori Elements 標準做法通常是彈出一個 **Message Popover**，或者存檔按鈕旁邊跳出一個訊息數量的紅色圓點圖示，點開會看到完整訊息列表）——訊息文字應該是「Title and Content must not be identical (Note ...)」，**Title / Content** 兩個欄位應該各自也帶著紅色錯誤標記
6. 把 **Content** 改成不一樣的文字，再按一次 **Create**，這次應該正常存檔成功

三種必填 / 驗證機制的完整對照

這一課實測下來，`ZI_RC05_NOTE` 這一個 Note 實體，`title` 欄位其實可以用三種完全不同的機制表達「這個欄位很重要」，行為差異很大，值得整理成一張表：

寫法	執行期真的會擋嗎	訊息文字	能不能跨欄位判斷
<code>field (mandatory)</code>	❌ 不會 (官方文件明講，實測也證實：無紅框、無提示、照樣存檔)	無	不行
<code>field (mandatory:create)</code>	✅ 會 (官方文件：「a runtime error occurs」)	系統制式文字，不能自訂	不行，只能單欄位判斷是否為空
自訂 <code>validation + %msg</code>	✅ 會 (<code>failed / reported</code> 明確擋下 Save)	完全自訂 (<code>new_message_with_text</code> 或 Message Class)	可以，這一課的 <code>validateTitleContent</code> 就是跨 <code>title / content</code> 兩欄位比對

打開瀏覽器開發者工具的 Network 分頁，比較後兩種被擋下時的請求 / 回應——這是驗證「這個錯誤到底是不是真的呼叫到後端商業邏輯」最直接的方法，不能只看畫面上有沒有紅框跟訊息文字判斷。

延伸：這個訊息彈窗跟 `sap.m.MessageToast` 是不是同一回事？

上面「動手操作」看到的紅色計數徽章 + 「Message Details」彈窗，跟 fe05 / fe06 講義裡出現過的 `MessageToast` (例如「已掛上標準 Demo Travel App」那種提示文字)，是兩套完全不同等級的機制，容易被表面上都是「跳出一段文字」誤認成同一種東西：

	<code>sap.m.MessageToast</code>	這一課看到的 Message Popover / Message Dialog
誰觸發的	開發者自己在程式碼裡呼叫 <code>MessageToast.show(...)</code>	框架自動接手處理 RAP 回傳的 <code>reported / failed</code> ，開發者完全不用 (也不能) 自己呼叫
呈現位置	呼叫時可自訂 (<code>at / my / of / offset</code> 錨定座標)	框架寫死錨定在特定位置 (footer 的 Message Button / 出錯欄位旁的紅框)，開發者無法傳參數調整
用途	一次性、自動消失的輕量提示	結構化訊息清單，可依嚴重度篩選、可導覽到出錯欄位、可看長文說明

查證官方文件 [Using Messages](#) 確認框架的判斷規則：只有「剛好只有一則成功的 Transition Message」這個特例，框架才會自動退化成用 `MessageToast` 顯示；其餘情況 (多筆訊息、任何錯誤訊息、跟欄位綁定的驗證錯誤) 一律走 Message Popover (Edit 模式，footer 的 Message Button) 或 Message Dialog (List Report / Display 模式) ——這一課 `validateTitleContent` 擋下存檔時看到的畫面，就是走後面這條路徑，不是 `MessageToast`。

這代表「幫這個訊息彈窗調位置/樣式」不能比照 `MessageToast` 傳參數的方式做，因為它根本不是你程序化呼叫出來的控制項。真正能介入的是框架提供的擴充點，而不是「參數」：

- **sap.fe.core.controllerextensions.MessageHandler** 的 **showMessageDialog** 方法——可以在 Controller Extension (fe05 已經教過這個機制) 裡覆寫，攔截 Transition Message 改成自己想要的呈現方式
- **ObjectPage.ExtensionAPI.showMessage()**——控制 Object Page 上方那條依嚴重度變色的 Message Strip 要不要顯示、顯示什麼
- **MessageButton Building Block**——如果是 fe04 教過的 Custom Page，這個訊息按鈕是自己擺進 XML View 版面的一般控制項，這種情況下位置才真的由開發者決定

學習目標

- 能講出 **field(mandatory)** 跟 **field(mandatory:create)** 的關鍵差異：前者官方文件明講「沒有執行期檢查」，只是消費端的提示旗標，這個 Fiori Elements 版本連視覺提示都沒有明顯呈現，實測完全不擋存檔；後者才是真正會擋下 Create 的寫法——這是本課實測親手推翻原始預期才挖出的重點，寫 BDEF 時不能望文生義
- 能講出三種「這個欄位很重要」機制的完整光譜：**mandatory** (僅提示，不擋) / **mandatory:create** (擋，但訊息不能自訂、不能跨欄位) / 自訂 **validation** (擋，訊息與邏輯完全自訂)
- 能寫出 **new_message_with_text(severity = ... text = ...)** 組出一個臨時 RAP 訊息，知道這是 **IF_ABAP_BEHV_MESSAGE** 介面提供的內建方法，不用自己寫 Message Class
- 知道 **%element-<欄位名> = if_abap_behv=>mk-on** 可以把錯誤標記釘在特定欄位上，一次可以標記多個欄位
- 知道 **new_message_with_text** (臨時文字，零額外物件) 跟 Message Class (**NEW zcx_...**，支援多語系 / 可重用) 的取捨，能講出各自適合的情境
- 能用瀏覽器開發者工具的 Network 分頁，分辨「這個錯誤有沒有真的打到後端」
- 能分辨 **sap.m.MessageToast** (開發者自己呼叫、可傳參數調位置/樣式) 跟框架自動處理的 Message Popover / Message Dialog (框架寫死呈現方式，只能透過 **MessageHandler** / **ExtensionAPI** 等擴充點介入，不是傳參數) ——知道兩者只有「剛好一則成功的 Transition Message」這個特例會重疊

物件清單

物件	型別	這一課的異動
ZI_RC05_NOTE	BDEF	① field (mandatory) title; 改成 field (mandatory:create) title; (Part A，見上) ② 新增 validation validateTitleContent on save { field title, content; } (Part B)
ZBP_I_RC05_NOTE	CLAS (Behavior Pool，Local Types)	第一次補上內容—— lhc_note 類別 + validateTitleContent Handler Method

沒有新建任何物件，套件沿用既有的 **ZRAPCLOUD** (非 **\$TMP**，寫入需要傳輸請求)。

動手練習

1. 想一想：如果把 **validation validateTitleContent on save { field title, content; }** 的觸發條件改成 **on save { create; }** (不管有沒有動到 **title/content**，每次 CREATE 都跑一次)，跟現在的 **field title, content;** 寫法，行為上會有什麼差異？ (提示：回顧 rc03 已經問過同一種問題，這裡換到你自已新寫的 Validation 上再想一次)

2. 試著把訊息換成 Info 等級 (`severity = if_abap_behv_message=>severity-information`)，存檔的行為會不會改變？ (提示：只有 Error 等級才會真的擋下存檔，Warning / Info 通常只是提示，不阻擋——實測驗證你的猜測)
3. 承 fe13 思考題 2：如果要讓 `status` 欄位「不能手動打清單以外的值」，現在你已經學過怎麼寫 Validation，試著自己設計一條規則 (提示：合法值只有 'O' / 'D'，rc03 的 `validateStatus` 剛好就是做這件事，你可以直接讀 `zi_rc01_task.bdef.abap` / Handler Method 對照)

驗證方式

✅ **Part A 前半 (`field(mandatory)` 不擋存檔)** 已由使用者實機驗證確認 (2026-08-24，`note_id = 'RC05TEST03'`，Title/Content 皆空白仍存檔成功，畫面全程無任何必填提示)——這個結果推翻了本課最早的草稿預期，已重寫成上面「實測推翻原本預期」的版本，並補上官方文件引用解釋原因。

✅ **Part A 後半 (`field(mandatory:create)` 會讓框架跳出 **Create Dialog**)** 已由使用者實機驗證確認 (2026-08-24，`title` 欄位帶紅色星號 *，List Report 點 Create 後不再直接進 Object Page，改跳只問 `note_id / title` 的小對話框)——這個發現比原始講義預期的「存檔被擋下」更細緻，已補進上面「實測確認」段落。

✅ **Part B (自訂 Validation，`new_message_with_text + %element` 錯誤標記)** 已由使用者實機驗證確認 (2026-08-24，`note_id = 'TestB'`，Title / Content 都打 **TitleB** 觸發驗證)：

1. `ZI_RC05_NOTE / ZBP_I_RC05_NOTE` 在 Eclipse 啟用成功 (過程中抓到並修正了 `LOOP AT ... WHERE title = content` 的語法錯誤，見上面「實測踩坑」段落)
2. 畫面跳出 **Message Details** 彈出視窗，訊息文字正確顯示「Title and Content must not be identical (Note Test...)」 (含動態嵌入的 `note_id`)
3. **Title / Content** 兩個欄位都出現紅框 (粉紅色錯誤外框)，證實 `%element-title / %element-content` 兩個標記都正確生效，不是只標一個
4. 畫面左下角出現錯誤計數徽章 (1)，存檔動作被擋下 (Draft 停在 **Draft updated** 狀態，**Create** 按鈕還在但尚未真正送出成功)

這一課從最早的草稿到實機驗證，中途推翻了一次錯誤假設 (`field(mandatory)` 不擋存檔)，抓到一個真正的 ABAP 語法錯誤 (`LOOP ... WHERE` 不能比較同一張表的兩個欄位)，最終 Part A / Part B 全部端對端驗證成功。

思考題

1. 這一課的 `validateTitleContent` 用 `IF ls_note-title = ls_note-content AND ls_note-title IS NOT INITIAL` 排除空字串情況 (避免 Title / Content 都空白時被誤判成「相同」)——這一課實測證實 `field(mandatory)` 本身根本不擋空白 Title，所以這個 `IS NOT INITIAL` 防呆條件其實比原本設計時想的更重要，不能假設 Title 一定有值。如果拿掉 `AND ls_note-title IS NOT INITIAL` 這個條件，你覺得會出現什麼副作用？ (提示：一筆 Title / Content 都留空的 Note，會不會也被 `validateTitleContent` 誤判成「相同」而擋下來？)
2. `%element-title / %element-content` 只是把錯誤標記「貼」在欄位上，不會阻止你在 Object Page 把這兩個欄位改成別的值再試一次。如果使用者只改了 `content` (讓兩者不再相同) 就重新存檔，這條 Validation 的觸發條件 (`field title, content;`) 確保它一定會重新跑一次嗎？
3. 對照 fe13 學到的「Value Help 只是建議清單，不是強制檢核」——這一課的 Validation 才是真正的「強制檢核」。如果你要幫 `status` 欄位同時做到「有下拉建議」+「不能亂打」，這兩課教的兩個機制要怎麼組合使用？

答案

見 `ZI_RC05_NOTE` (`BDEF` , `field(mandatory) title` 改成 `field(mandatory:create) title` + 新增 `validation` 那一行) 、 `ZBP_I_RC05_NOTE` (`Local Types` , `lhc_note` 類別完整內容如上 , 含修正過 `LOOP ... WHERE` 語法錯誤的版本) 。前端沒有新增或修改任何檔案 , 完全沿用 `fe01_connection_test` 既有專案。**Part A / Part B 全部端對端實機驗證成功** (2026-08-24) ——過程中推翻了一次錯誤假設 (`field(mandatory)` 不擋存檔) 、抓到並修正一個真正的 ABAP 語法錯誤 (`LOOP ... WHERE` 不能比較同一張表的兩個欄位) , 也修正了兩處把本機快照檔名誤植成 Eclipse 開啟指令的講義措辭——這一課完整走過「設計→實測推翻→查證→修正→再驗證」的完整迴圈 , 是這門延伸篇目前最扎實的一課。